23. C. J. H. McDiarmid and B. A. Reed, "Building Heaps Fast," *Journal of Algorithms,* 10 (1989), 352–365.
24. D. D. Sleator and R. E. Tarjan, "Self-adjusting Heaps," *SIAM Journal on Computing,* 15 (1986), 52–69.
25. T. Strothotte, P. Eriksson, and S. Vallner, "A Note on Constructing Min-max Heaps," *BIT,* 29 (1989), 251–256.
26. P. van Emde Boas, R. Kaas, and E. Zijlstra, "Design and Implementation of an Efficient Priority Queue," *Mathematical Systems Theory,* 10 (1977), 99–127.
27. J. Vuillemin, "A Data Structure for Manipulating Priority Queues," *Communications of the ACM,* 21 (1978), 309–314.
28. J. W. J. Williams, "Algorithm 232: Heapsort," *Communications of the ACM,* 7 (1964), 347–348.

CHAPTER 7

Sorting

In this chapter, we discuss the problem of sorting an array of elements. To simplify matters, we will assume in our examples that the array contains only integers, although our code will once again allow more general objects. For most of this chapter, we will also assume that the entire sort can be done in main memory, so that the number of elements is relatively small (less than a few million). Sorts that cannot be performed in main memory and must be done on disk or tape are also quite important. This type of sorting, known as external sorting, will be discussed at the end of the chapter.

Our investigation of internal sorting will show that...

- There are several easy algorithms to sort in $O(N^2)$, such as insertion sort.
- There is an algorithm, Shellsort, that is very simple to code, runs in $o(N^2)$, and is efficient in practice.
- There are slightly more complicated $O(N \log N)$ sorting algorithms.
- Any general-purpose sorting algorithm requires $\Omega(N \log N)$ comparisons.

The rest of this chapter will describe and analyze the various sorting algorithms. These algorithms contain interesting and important ideas for code optimization as well as algorithm design. Sorting is also an example where the analysis can be precisely performed. Be forewarned that where appropriate, we will do as much analysis as possible.

7.1 Preliminaries

The algorithms we describe will all be interchangeable. Each will be passed an array containing the elements; we assume all array positions contain data to be sorted. We will assume that N is the number of elements passed to our sorting routines.

We will also assume the existence of the "<" and ">" operators, which can be used to place a consistent ordering on the input. Besides the assignment operator, these are the only operations allowed on the input data. Sorting under these conditions is known as **comparison-based sorting**.

This interface is not the same as in the STL sorting algorithms. In the STL, sorting is accomplished by use of the function template `sort`. The parameters to `sort` represent the start and endmarker of a (range in a) container and an optional comparator:

```
void sort( Iterator begin, Iterator end );
void sort( Iterator begin, Iterator end, Comparator cmp );
```

The iterators must support random access. The `sort` algorithm does not guarantee that equal items retain their original order (if that is important, use `stable_sort` instead of `sort`).

As an example, in

```
std::sort( v.begin( ), v.end( ) );
std::sort( v.begin( ), v.end( ), greater<int>{ } );
std::sort( v.begin( ), v.begin( ) + ( v.end( ) - v.begin( ) ) / 2 );
```

the first call sorts the entire container, `v`, in nondecreasing order. The second call sorts the entire container in nonincreasing order. The third call sorts the first half of the container in nondecreasing order.

The sorting algorithm used is generally quicksort, which we describe in Section 7.7. In Section 7.2, we implement the simplest sorting algorithm using both our style of passing the array of comparable items, which yields the most straightforward code, and the interface supported by the STL, which requires more code.

7.2 Insertion Sort

One of the simplest sorting algorithms is the **insertion sort**.

7.2.1 The Algorithm

Insertion sort consists of $N-1$ **passes.** For pass $p=1$ through $N-1$, insertion sort ensures that the elements in positions 0 through p are in sorted order. Insertion sort makes use of the fact that elements in positions 0 through $p-1$ are already known to be in sorted order. Figure 7.1 shows a sample array after each pass of insertion sort.

Figure 7.1 shows the general strategy. In pass p, we move the element in position p left until its correct place is found among the first $p+1$ elements. The code in Figure 7.2 implements this strategy. Lines 11 to 14 implement that data movement without the explicit use of swaps. The element in position p is moved to `tmp`, and all larger elements (prior to position p) are moved one spot to the right. Then `tmp` is moved to the correct spot. This is the same technique that was used in the implementation of binary heaps.

Original	34	8	64	51	32	21	Positions Moved
After $p=1$	8	34	64	51	32	21	1
After $p=2$	8	34	64	51	32	21	0
After $p=3$	8	34	51	64	32	21	1
After $p=4$	8	32	34	51	64	21	3
After $p=5$	8	21	32	34	51	64	4

Figure 7.1 Insertion sort after each pass

```
1    /**
2     * Simple insertion sort.
3     */
4    template <typename Comparable>
5    void insertionSort( vector<Comparable> & a )
6    {
7        for( int p = 1; p < a.size( ); ++p )
8        {
9            Comparable tmp = std::move( a[ p ] );
10
11           int j;
12           for( j = p; j > 0 && tmp < a[ j - 1 ]; --j )
13               a[ j ] = std::move( a[ j - 1 ] );
14           a[ j ] = std::move( tmp );
15       }
16   }
```

Figure 7.2 Insertion sort routine

7.2.2 STL Implementation of Insertion Sort

In the STL, instead of having the sort routines take an array of comparable items as a single parameter, the sort routines receive a pair of iterators that represent the start and endmarker of a range. A two-parameter sort routine uses just that pair of iterators and presumes that the items can be ordered, while a three-parameter sort routine has a function object as a third parameter.

Converting the algorithm in Figure 7.2 to use the STL introduces several issues. The obvious issues are

1. We must write a two-parameter sort and a three-parameter sort. Presumably, the two-parameter sort invokes the three-parameter sort, with `less<Object>{ }` as the third parameter.
2. Array access must be converted to iterator access.
3. Line 11 of the original code requires that we create `tmp`, which in the new code will have type `Object`.

The first issue is the trickiest because the template type parameters (i.e., the generic types) for the two-parameter sort are both `Iterator`; however, `Object` is not one of the generic type parameters. Prior to C++11, one had to write extra routines to solve this problem. As shown in Figure 7.3, C++11 introduces `decltype` which cleanly expresses the intent.

Figure 7.4 shows the main sorting code that replaces array indexing with use of the iterator, and that replaces calls to `operator<` with calls to the `lessThan` function object.

Observe that once we actually code the `insertionSort` algorithm, every statement in the original code is replaced with a corresponding statement in the new code that makes

```
/*
 * The two-parameter version calls the three-parameter version,
 * using C++11 decltype
 */
template <typename Iterator>
void insertionSort( const Iterator & begin, const Iterator & end )
{
    insertionSort( begin, end, less<decltype(*begin)>{ } );
}
```

Figure 7.3 Two-parameter sort invokes three-parameter sort via C++11 `decltype`

```
template <typename Iterator, typename Comparator>
void insertionSort( const Iterator & begin, const Iterator & end,
                    Comparator lessThan )
{
    if( begin == end )
        return;

    Iterator j;

    for( Iterator p = begin+1; p != end; ++p )
    {
        auto tmp = std::move( *p );
        for( j = p; j != begin && lessThan( tmp, *( j-1 ) ); --j )
            *j = std::move( *(j-1) );
        *j = std::move( tmp );
    }
}
```

Figure 7.4 Three-parameter sort using iterators

straightforward use of iterators and the function object. The original code is arguably much simpler to read, which is why we use our simpler interface rather than the STL interface when coding our sorting algorithms.

7.2.3 Analysis of Insertion Sort

Because of the nested loops, each of which can take N iterations, insertion sort is $O(N^2)$. Furthermore, this bound is tight, because input in reverse order can achieve this bound. A precise calculation shows that the number of tests in the inner loop in Figure 7.2 is at most $p + 1$ for each value of p. Summing over all p gives a total of

$$\sum_{i=2}^{N} i = 2 + 3 + 4 + \cdots + N = \Theta(N^2)$$

On the other hand, if the input is presorted, the running time is $O(N)$, because the test in the inner `for` loop always fails immediately. Indeed, if the input is almost sorted (this term will be more rigorously defined in the next section), insertion sort will run quickly. Because of this wide variation, it is worth analyzing the average-case behavior of this algorithm. It turns out that the average case is $\Theta(N^2)$ for insertion sort, as well as for a variety of other sorting algorithms, as the next section shows.

7.3 A Lower Bound for Simple Sorting Algorithms

An **inversion** in an array of numbers is any ordered pair (i, j) having the property that $i < j$ but $a[i] > a[j]$. In the example of the last section, the input list 34, 8, 64, 51, 32, 21 had nine inversions, namely $(34, 8)$, $(34, 32)$, $(34, 21)$, $(64, 51)$, $(64, 32)$, $(64, 21)$, $(51, 32)$, $(51, 21)$, and $(32, 21)$. Notice that this is exactly the number of swaps that needed to be (implicitly) performed by insertion sort. This is always the case, because swapping two adjacent elements that are out of place removes exactly one inversion, and a sorted array has no inversions. Since there is $O(N)$ other work involved in the algorithm, the running time of insertion sort is $O(I + N)$, where I is the number of inversions in the original array. Thus, insertion sort runs in linear time if the number of inversions is $O(N)$.

We can compute precise bounds on the average running time of insertion sort by computing the average number of inversions in a permutation. As usual, defining *average* is a difficult proposition. We will assume that there are no duplicate elements (if we allow duplicates, it is not even clear what the average number of duplicates is). Using this assumption, we can assume that the input is some permutation of the first N integers (since only relative ordering is important) and that all are equally likely. Under these assumptions, we have the following theorem:

Theorem 7.1
The average number of inversions in an array of N distinct elements is $N(N - 1)/4$.

Proof
For any list, L, of elements, consider L_r, the list in reverse order. The reverse list of the example is $21, 32, 51, 64, 8, 34$. Consider any pair of two elements in the list (x, y) with $y > x$. Clearly, in exactly one of L and L_r this ordered pair represents an inversion. The total number of these pairs in a list L and its reverse L_r is $N(N-1)/2$. Thus, an average list has half this amount, or $N(N - 1)/4$ inversions.

This theorem implies that insertion sort is quadratic on average. It also provides a very strong lower bound about any algorithm that only exchanges adjacent elements.

Theorem 7.2
Any algorithm that sorts by exchanging adjacent elements requires $\Omega(N^2)$ time on average.

Proof

The average number of inversions is initially $N(N-1)/4 = \Omega(N^2)$. Each swap removes only one inversion, so $\Omega(N^2)$ swaps are required.

This is an example of a lower-bound proof. It is valid not only for insertion sort, which performs adjacent exchanges implicitly, but also for other simple algorithms such as bubble sort and selection sort, which we will not describe here. In fact, it is valid over an entire *class* of sorting algorithms, including those undiscovered, that perform only adjacent exchanges. Because of this, this proof cannot be confirmed empirically. Although this lower-bound proof is rather simple, in general proving lower bounds is much more complicated than proving upper bounds and in some cases resembles magic.

This lower bound shows us that in order for a sorting algorithm to run in subquadratic, or $o(N^2)$, time, it must do comparisons and, in particular, exchanges between elements that are far apart. A sorting algorithm makes progress by eliminating inversions, and to run efficiently, it must eliminate more than just one inversion per exchange.

7.4 Shellsort

Shellsort, named after its inventor, Donald Shell, was one of the first algorithms to break the quadratic time barrier, although it was not until several years after its initial discovery that a subquadratic time bound was proven. As suggested in the previous section, it works by comparing elements that are distant; the distance between comparisons decreases as the algorithm runs until the last phase, in which adjacent elements are compared. For this reason, Shellsort is sometimes referred to as **diminishing increment sort**.

Shellsort uses a sequence, $h_1, h_2, \ldots, h_t$, called the **increment sequence**. Any increment sequence will do as long as $h_1 = 1$, but some choices are better than others (we will discuss that issue later). After a *phase,* using some increment h_k, for every i, we have $a[i] \leq a[i + h_k]$ (where this makes sense); all elements spaced h_k apart are sorted. The file is then said to be **h_k-sorted.** For example, Figure 7.5 shows an array after several phases of Shellsort. An important property of Shellsort (which we state without proof) is that an h_k-sorted file that is then h_{k-1}-sorted remains h_k-sorted. If this were not the case, the algorithm would likely be of little value, since work done by early phases would be undone by later phases.

The general strategy to h_k-sort is for each position, i, in $h_k, h_k + 1, \ldots, N - 1$, place the element in the correct spot among $i, i - h_k, i - 2h_k$, and so on. Although this does not

Original	81	94	11	96	12	35	17	95	28	58	41	75	15
After 5-sort	35	17	11	28	12	41	75	15	96	58	81	94	95
After 3-sort	28	12	11	35	15	41	58	17	94	75	81	96	95
After 1-sort	11	12	15	17	28	35	41	58	75	81	94	95	96

Figure 7.5 Shellsort after each pass

```
/**
 * Shellsort, using Shell's (poor) increments.
 */
template <typename Comparable>
void shellsort( vector<Comparable> & a )
{
    for( int gap = a.size( ) / 2; gap > 0; gap /= 2 )
        for( int i = gap; i < a.size( ); ++i )
        {
            Comparable tmp = std::move( a[ i ] );
            int j = i;

            for( ; j >= gap && tmp < a[ j - gap ]; j -= gap )
                a[ j ] = std::move( a[ j - gap ] );
            a[ j ] = std::move( tmp );
        }
}
```

Figure 7.6 Shellsort routine using Shell's increments (better increments are possible)

affect the implementation, a careful examination shows that the action of an h_k-sort is to perform an insertion sort on h_k independent subarrays. This observation will be important when we analyze the running time of Shellsort.

A popular (but poor) choice for increment sequence is to use the sequence suggested by Shell: $h_t = \lfloor N/2 \rfloor$, and $h_k = \lfloor h_{k+1}/2 \rfloor$. Figure 7.6 contains a function that implements Shellsort using this sequence. We shall see later that there are increment sequences that give a significant improvement in the algorithm's running time; even a minor change can drastically affect performance (Exercise 7.10).

The program in Figure 7.6 avoids the explicit use of swaps in the same manner as our implementation of insertion sort.

7.4.1 Worst-Case Analysis of Shellsort

Although Shellsort is simple to code, the analysis of its running time is quite another story. The running time of Shellsort depends on the choice of increment sequence, and the proofs can be rather involved. The average-case analysis of Shellsort is a long-standing open problem, except for the most trivial increment sequences. We will prove tight worst-case bounds for two particular increment sequences.

Theorem 7.3

The worst-case running time of Shellsort using Shell's increments is $\Theta(N^2)$.

Proof

The proof requires showing not only an upper bound on the worst-case running time but also showing that there exists some input that actually takes $\Omega(N^2)$ time to run.

We prove the lower bound first by constructing a bad case. First, we choose N to be a power of 2. This makes all the increments even, except for the last increment, which is 1. Now, we will give as input an array with the $N/2$ largest numbers in the even positions and the $N/2$ smallest numbers in the odd positions (for this proof, the first position is position 1). As all the increments except the last are even, when we come to the last pass, the $N/2$ largest numbers are still all in even positions and the $N/2$ smallest numbers are still all in odd positions. The ith smallest number ($i \leq N/2$) is thus in position $2i - 1$ before the beginning of the last pass. Restoring the ith element to its correct place requires moving it $i-1$ spaces in the array. Thus, to merely place the $N/2$ smallest elements in the correct place requires at least $\sum_{i=1}^{N/2} i - 1 = \Omega(N^2)$ work. As an example, Figure 7.7 shows a bad (but not the worst) input when $N = 16$. The number of inversions remaining after the 2-sort is exactly $1+2+3+4+5+6+7 = 28$; thus, the last pass will take considerable time.

To finish the proof, we show the upper bound of $O(N^2)$. As we have observed before, a pass with increment h_k consists of h_k insertion sorts of about N/h_k elements. Since insertion sort is quadratic, the total cost of a pass is $O(h_k(N/h_k)^2) = O(N^2/h_k)$. Summing over all passes gives a total bound of $O(\sum_{i=1}^{t} N^2/h_i) = O(N^2 \sum_{i=1}^{t} 1/h_i)$. Because the increments form a geometric series with common ratio 2, and the largest term in the series is $h_1 = 1$, $\sum_{i=1}^{t} 1/h_i < 2$. Thus we obtain a total bound of $O(N^2)$.

The problem with Shell's increments is that pairs of increments are not necessarily relatively prime, and thus the smaller increment can have little effect. Hibbard suggested a slightly different increment sequence, which gives better results in practice (and theoretically). His increments are of the form $1, 3, 7, \ldots, 2^k - 1$. Although these increments are almost identical, the key difference is that consecutive increments have no common factors. We now analyze the worst-case running time of Shellsort for this increment sequence. The proof is rather complicated.

Theorem 7.4

The worst-case running time of Shellsort using Hibbard's increments is $\Theta(N^{3/2})$.

Proof

We will prove only the upper bound and leave the proof of the lower bound as an exercise. The proof requires some well-known results from additive number theory. References to these results are provided at the end of the chapter.

For the upper bound, as before, we bound the running time of each pass and sum over all passes. For increments $h_k > N^{1/2}$, we will use the bound $O(N^2/h_k)$ from the

Start	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
After 8-sort	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
After 4-sort	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
After 2-sort	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
After 1-sort	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Figure 7.7 Bad case for Shellsort with Shell's increments (positions are numbered 1 to 16)

previous theorem. Although this bound holds for the other increments, it is too large to be useful. Intuitively, we must take advantage of the fact that *this* increment sequence is *special*. What we need to show is that for any element $a[p]$ in position p, when it is time to perform an h_k-sort, there are only a few elements to the left of position p that are larger than $a[p]$.

When we come to h_k-sort the input array, we know that it has already been h_{k+1}- and h_{k+2}-sorted. Prior to the h_k-sort, consider elements in positions p and $p - i$, $i \leq p$. If i is a multiple of h_{k+1} or h_{k+2}, then clearly $a[p - i] < a[p]$. We can say more, however. If i is expressible as a linear combination (in nonnegative integers) of h_{k+1} and h_{k+2}, then $a[p - i] < a[p]$. As an example, when we come to 3-sort, the file is already 7- and 15-sorted. 52 is expressible as a linear combination of 7 and 15, because $52 = 1 * 7 + 3 * 15$. Thus, $a[100]$ cannot be larger than $a[152]$ because $a[100] \leq a[107] \leq a[122] \leq a[137] \leq a[152]$.

Now, $h_{k+2} = 2h_{k+1} + 1$, so h_{k+1} and h_{k+2} cannot share a common factor. In this case, it is possible to show that all integers that are at least as large as $(h_{k+1} - 1)(h_{k+2} - 1) = 8h_k^2 + 4h_k$ can be expressed as a linear combination of h_{k+1} and h_{k+2} (see the reference at the end of the chapter).

This tells us that the body of the innermost `for` loop can be executed at most $8h_k + 4 = O(h_k)$ times for each of the $N - h_k$ positions. This gives a bound of $O(Nh_k)$ per pass.

Using the fact that about half the increments satisfy $h_k < \sqrt{N}$, and assuming that t is even, the total running time is then

$$O\left(\sum_{k=1}^{t/2} Nh_k + \sum_{k=t/2+1}^{t} N^2/h_k\right) = O\left(N\sum_{k=1}^{t/2} h_k + N^2 \sum_{k=t/2+1}^{t} 1/h_k\right)$$

Because both sums are geometric series, and since $h_{t/2} = \Theta(\sqrt{N})$, this simplifies to

$$= O\left(Nh_{t/2}\right) + O\left(\frac{N^2}{h_{t/2}}\right) = O(N^{3/2})$$

The average-case running time of Shellsort, using Hibbard's increments, is thought to be $O(N^{5/4})$, based on simulations, but nobody has been able to prove this. Pratt has shown that the $\Theta(N^{3/2})$ bound applies to a wide range of increment sequences.

Sedgewick has proposed several increment sequences that give an $O(N^{4/3})$ worst-case running time (also achievable). The average running time is conjectured to be $O(N^{7/6})$ for these increment sequences. Empirical studies show that these sequences perform significantly better in practice than Hibbard's. The best of these is the sequence $\{1, 5, 19, 41, 109, \ldots\}$, in which the terms are either of the form $9 \cdot 4^i - 9 \cdot 2^i + 1$ or $4^i - 3 \cdot 2^i + 1$. This is most easily implemented by placing these values in an array. This increment sequence is the best known in practice, although there is a lingering possibility that some increment sequence might exist that could give a significant improvement in the running time of Shellsort.

There are several other results on Shellsort that (generally) require difficult theorems from number theory and combinatorics and are mainly of theoretical interest. Shellsort is a fine example of a very simple algorithm with an extremely complex analysis.

The performance of Shellsort is quite acceptable in practice, even for N in the tens of thousands. The simplicity of the code makes it the algorithm of choice for sorting up to moderately large input.

7.5 Heapsort

As mentioned in Chapter 6, priority queues can be used to sort in $O(N \log N)$ time. The algorithm based on this idea is known as **heapsort** and gives the best Big-Oh running time we have seen so far.

Recall from Chapter 6 that the basic strategy is to build a binary heap of N elements. This stage takes $O(N)$ time. We then perform N `deleteMin` operations. The elements leave the heap smallest first, in sorted order. By recording these elements in a second array and then copying the array back, we sort N elements. Since each `deleteMin` takes $O(\log N)$ time, the total running time is $O(N \log N)$.

The main problem with this algorithm is that it uses an extra array. Thus, the memory requirement is doubled. This could be a problem in some instances. Notice that the extra time spent copying the second array back to the first is only $O(N)$, so that this is not likely to affect the running *time* significantly. The problem is space.

A clever way to avoid using a second array makes use of the fact that after each `deleteMin`, the heap shrinks by 1. Thus the cell that was last in the heap can be used to store the element that was just deleted. As an example, suppose we have a heap with six elements. The first `deleteMin` produces a_1. Now the heap has only five elements, so we can place a_1 in position 6. The next `deleteMin` produces a_2. Since the heap will now only have four elements, we can place a_2 in position 5.

Using this strategy, after the last `deleteMin` the array will contain the elements in *decreasing* sorted order. If we want the elements in the more typical *increasing* sorted order, we can change the ordering property so that the parent has a larger element than the child. Thus, we have a (*max*)heap.

In our implementation, we will use a (*max*)heap but avoid the actual ADT for the purposes of speed. As usual, everything is done in an array. The first step builds the heap in linear time. We then perform $N - 1$ `deleteMax`es by swapping the last element in the heap with the first, decrementing the heap size, and percolating down. When the algorithm terminates, the array contains the elements in sorted order. For instance, consider the input sequence $31, 41, 59, 26, 53, 58, 97$. The resulting heap is shown in Figure 7.8.

Figure 7.9 shows the heap that results after the first `deleteMax`. As the figures imply, the last element in the heap is 31; 97 has been placed in a part of the heap array that is technically no longer part of the heap. After 5 more `deleteMax` operations, the heap will actually have only one element, but the elements left in the heap array will be in sorted order.

The code to perform heapsort is given in Figure 7.10. The slight complication is that, unlike the binary heap, where the data begin at array index 1, the array for heapsort contains data in position 0. Thus the code is a little different from the binary heap code. The changes are minor.

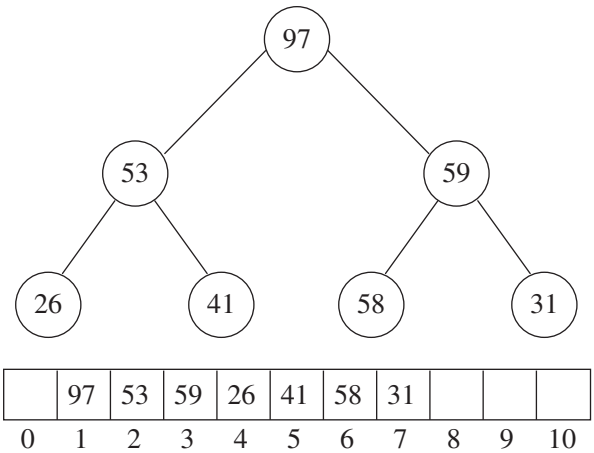


Figure 7.8 (*Max*) heap after `buildHeap` phase

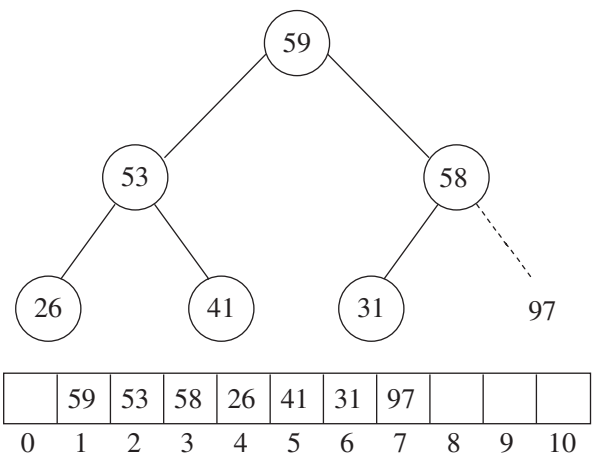


Figure 7.9 Heap after first `deleteMax`

7.5.1 Analysis of Heapsort

As we saw in Chapter 6, the first phase, which constitutes the building of the heap, uses less than $2N$ comparisons. In the second phase, the ith `deleteMax` uses at most less than $2\lfloor \log(N - i + 1) \rfloor$ comparisons, for a total of at most $2N \log N - O(N)$ comparisons (assuming $N \geq 2$). Consequently, in the worst case, at most $2N \log N - O(N)$ comparisons are used by heapsort. Exercise 7.13 asks you to show that it is possible for all of the `deleteMax` operations to achieve their worst case simultaneously.

```
/**
 * Standard heapsort.
 */
template <typename Comparable>
void heapsort( vector<Comparable> & a )
{
    for( int i = a.size( ) / 2 - 1; i >= 0; --i )  /* buildHeap */
        percDown( a, i, a.size( ) );
    for( int j = a.size( ) - 1; j > 0; --j )
    {
        std::swap( a[ 0 ], a[ j ] );               /* deleteMax */
        percDown( a, 0, j );
    }
}

/**
 * Internal method for heapsort.
 * i is the index of an item in the heap.
 * Returns the index of the left child.
 */
inline int leftChild( int i )
{
    return 2 * i + 1;
}

/**
 * Internal method for heapsort that is used in deleteMax and buildHeap.
 * i is the position from which to percolate down.
 * n is the logical size of the binary heap.
 */
template <typename Comparable>
void percDown( vector<Comparable> & a, int i, int n )
{
    int child;
    Comparable tmp;

    for( tmp = std::move( a[ i ] ); leftChild( i ) < n; i = child )
    {
        child = leftChild( i );
        if( child != n - 1 && a[ child ] < a[ child + 1 ] )
            ++child;
        if( tmp < a[ child ] )
            a[ i ] = std::move( a[ child ] );
        else
            break;
    }
    a[ i ] = std::move( tmp );
}
```

Figure 7.10 Heapsort

Experiments have shown that the performance of heapsort is extremely consistent: On average it uses only slightly fewer comparisons than the worst-case bound suggests. For many years, nobody had been able to show nontrivial bounds on heapsort's average running time. The problem, it seems, is that successive `deleteMax` operations destroy the heap's randomness, making the probability arguments very complex. Eventually, another approach proved successful.

Theorem 7.5
The average number of comparisons used to heapsort a random permutation of N distinct items is $2N \log N - O(N \log \log N)$.

Proof
The heap construction phase uses $\Theta(N)$ comparisons on average, and so we only need to prove the bound for the second phase. We assume a permutation of $\{1, 2, \ldots, N\}$.

Suppose the ith `deleteMax` pushes the root element down d_i levels. Then it uses $2d_i$ comparisons. For heapsort on any input, there is a cost sequence $D : d_1, d_2, \ldots, d_N$ that defines the cost of phase 2. That cost is given by $M_D = \sum_{i=1}^{N} d_i$; the number of comparisons used is thus $2M_D$.

Let $f(N)$ be the number of heaps of N items. One can show (Exercise 7.58) that $f(N) > (N/(4e))^N$ (where $e = 2.71828\ldots$). We will show that only an exponentially small fraction of these heaps (in particular $(N/16)^N$) have a cost smaller than $M = N(\log N - \log \log N - 4)$. When this is shown, it follows that the average value of M_D is at least M minus a term that is $o(1)$, and thus the average number of comparisons is at least $2M$. Consequently, our basic goal is to show that there are very few heaps that have small cost sequences.

Because level d_i has at most 2^{d_i} nodes, there are 2^{d_i} possible places that the root element can go for any d_i. Consequently, for any sequence D, the number of distinct corresponding `deleteMax` sequences is at most

$$S_D = 2^{d_1} 2^{d_2} \cdots 2^{d_N}$$

A simple algebraic manipulation shows that *for a given sequence* D,

$$S_D = 2^{M_D}$$

Because each d_i can assume any value between 1 and $\lfloor \log N \rfloor$, there are at most $(\log N)^N$ possible sequences D. It follows that the number of distinct `deleteMax` sequences that require cost exactly equal to M is at most the number of cost sequences of total cost M times the number of `deleteMax` sequences for each of these cost sequences. A bound of $(\log N)^N 2^M$ follows immediately.

The total number of heaps with cost sequence less than M is at most

$$\sum_{i=1}^{M-1} (\log N)^N 2^i < (\log N)^N 2^M$$

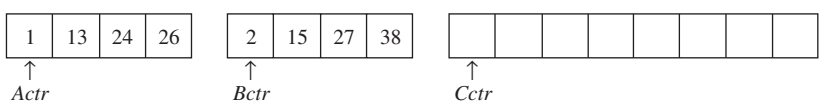
If we choose $M = N(\log N - \log\log N - 4)$, then the number of heaps that have cost sequence less than M is at most $(N/16)^N$, and the theorem follows from our earlier comments.

Using a more complex argument, it can be shown that heapsort always uses at least $N \log N - O(N)$ comparisons and that there are inputs that can achieve this bound. The average-case analysis also can be improved to $2N \log N - O(N)$ comparisons (rather than the nonlinear second term in Theorem 7.5).

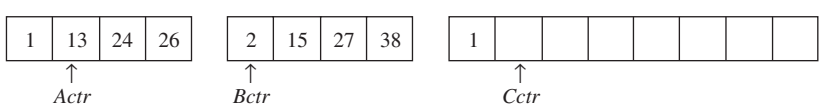
7.6 Mergesort

We now turn our attention to **mergesort**. Mergesort runs in $O(N \log N)$ worst-case running time, and the number of comparisons used is nearly optimal. It is a fine example of a recursive algorithm.

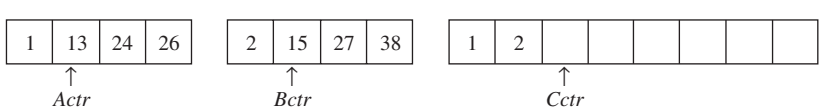
The fundamental operation in this algorithm is merging two sorted lists. Because the lists are sorted, this can be done in one pass through the input, if the output is put in a third list. The basic merging algorithm takes two input arrays A and B, an output array C, and three counters, *Actr*, *Bctr*, and *Cctr*, which are initially set to the beginning of their respective arrays. The smaller of $A[Actr]$ and $B[Bctr]$ is copied to the next entry in C, and the appropriate counters are advanced. When either input list is exhausted, the remainder of the other list is copied to C. An example of how the merge routine works is provided for the following input.



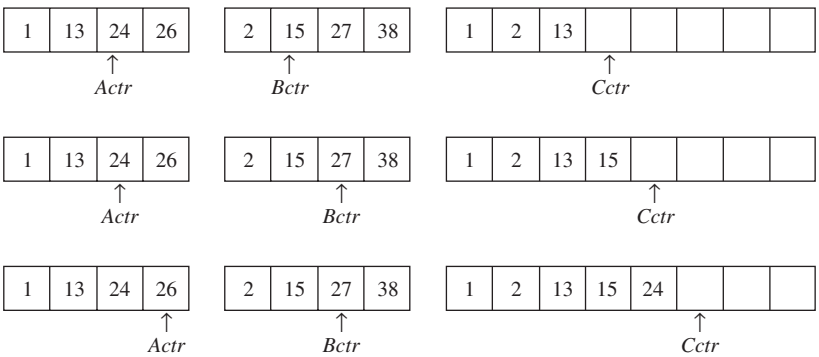
If the array A contains 1, 13, 24, 26, and B contains 2, 15, 27, 38, then the algorithm proceeds as follows: First, a comparison is done between 1 and 2. 1 is added to C, and then 13 and 2 are compared.



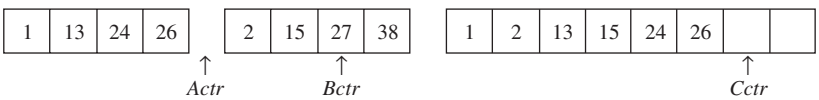
2 is added to C, and then 13 and 15 are compared.



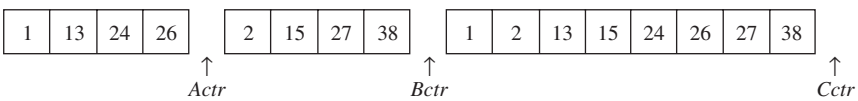
13 is added to C, and then 24 and 15 are compared. This proceeds until 26 and 27 are compared.



26 is added to C, and the A array is exhausted.



The remainder of the B array is then copied to C.



The time to merge two sorted lists is clearly linear, because at most $N - 1$ comparisons are made, where N is the total number of elements. To see this, note that every comparison adds an element to C, except the last comparison, which adds at least two.

The mergesort algorithm is therefore easy to describe. If $N = 1$, there is only one element to sort, and the answer is at hand. Otherwise, recursively mergesort the first half and the second half. This gives two sorted halves, which can then be merged together using the merging algorithm described above. For instance, to sort the eight-element array $24, 13, 26, 1, 2, 27, 38, 15$, we recursively sort the first four and last four elements, obtaining $1, 13, 24, 26, 2, 15, 27, 38$. Then we merge the two halves as above, obtaining the final list $1, 2, 13, 15, 24, 26, 27, 38$. This algorithm is a classic divide-and-conquer strategy. The problem is *divided* into smaller problems and solved recursively. The *conquering* phase consists of patching together the answers. Divide-and-conquer is a very powerful use of recursion that we will see many times.

An implementation of mergesort is provided in Figure 7.11. The one-parameter `mergeSort` is just a driver for the four-parameter recursive `mergeSort`.

The `merge` routine is subtle. If a temporary array is declared locally for each recursive call of `merge`, then there could be $\log N$ temporary arrays active at any point. A close examination shows that since `merge` is the last line of `mergeSort`, there only needs to be one